

Characterizing IR Generated by rustc and clang

Benjamin Ye, Tommy M^cMichen

Rust’s abstractions add rich information for analysis

Implements equality

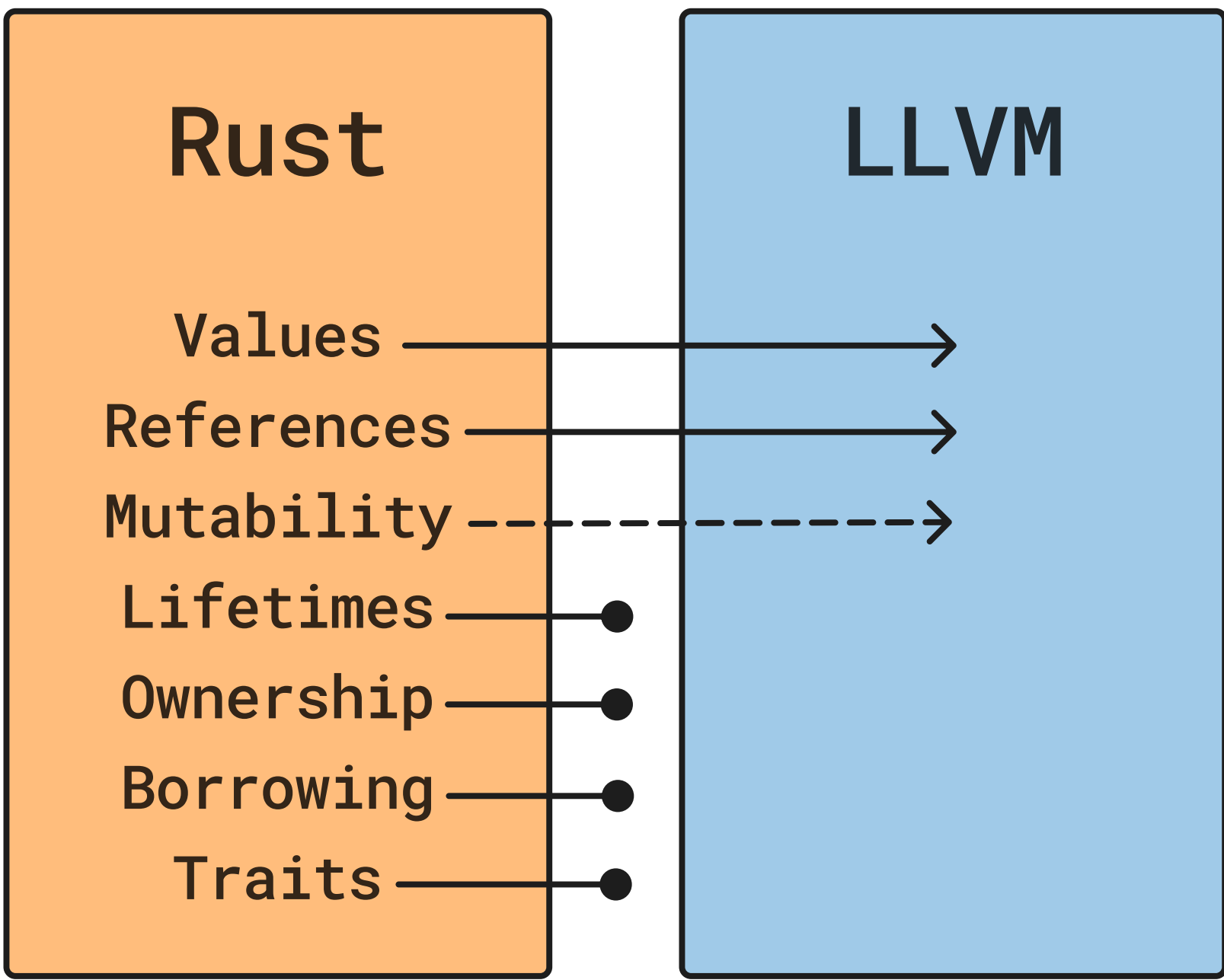
Singular mutable ref

fn run<T: Eq>(foo: &T, bar: &mut T) {
 ...
}

Shareable immutable ref

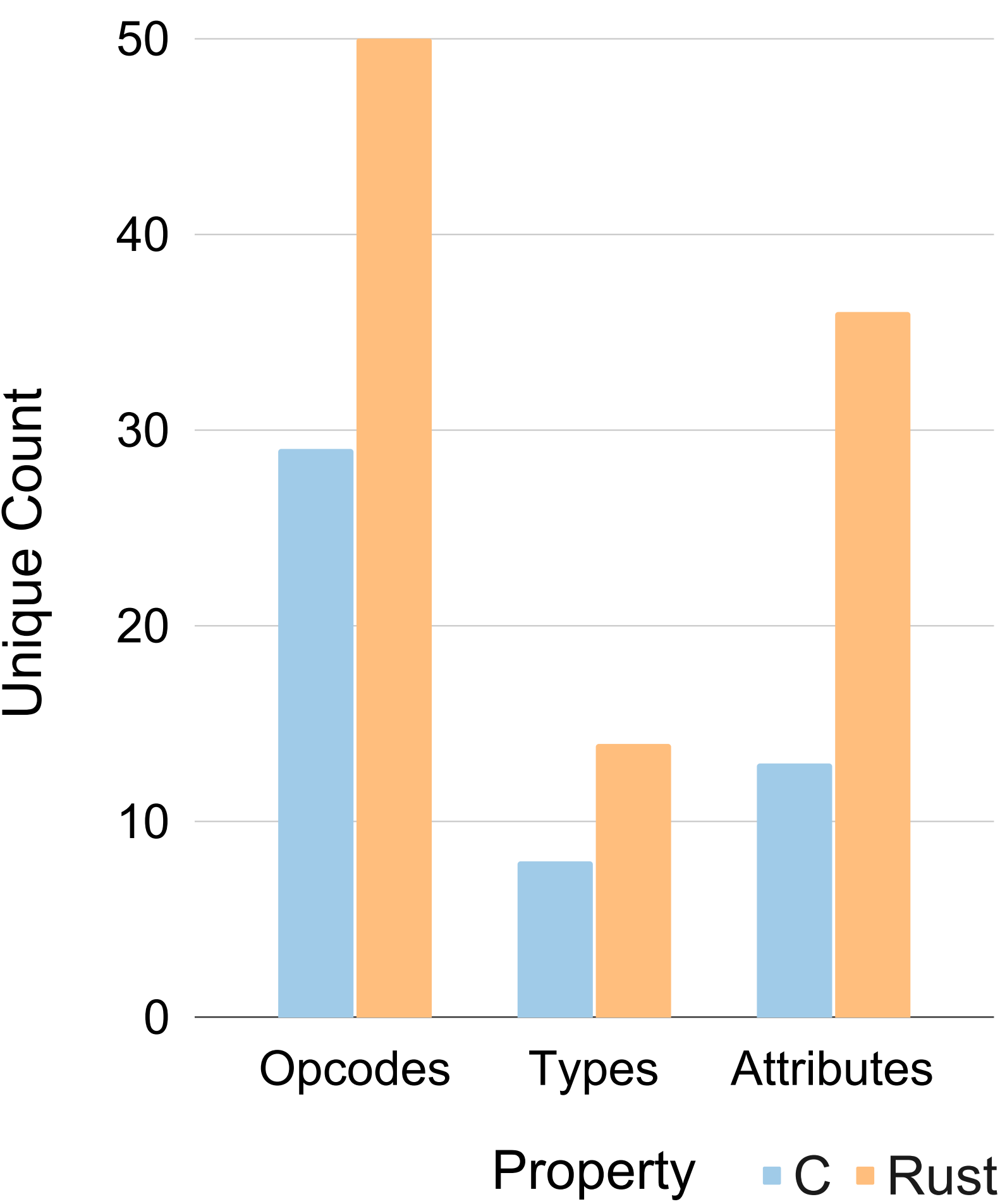
Encodes guarantees about aliasing and usage!

A shared IR wastes that information



Do programs in different languages generate different IR?

Some information is preserved!



While not explicit, Rust’s guarantees show up in IR structure

Def-Use Graph

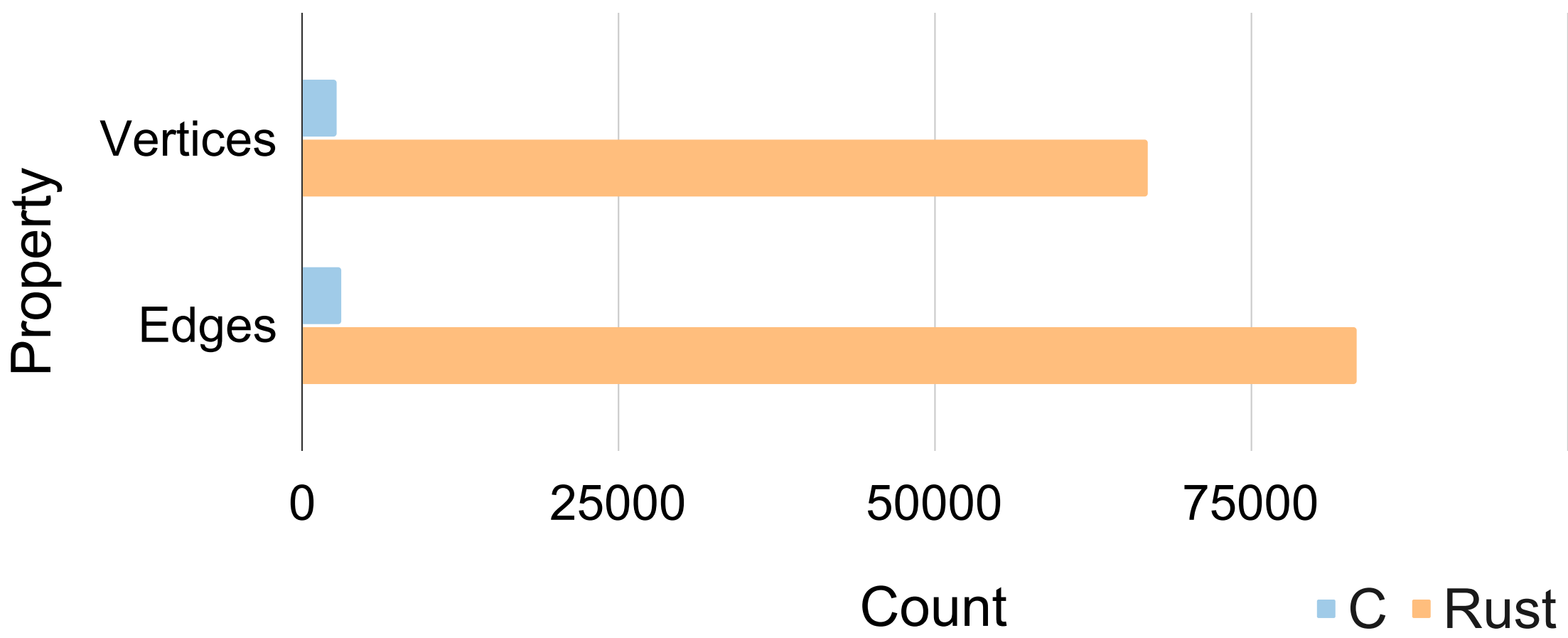
c = a + b;
d = b - c;
e = a * a;

Treewidth: how far a graph is from a tree

Pathwidth: how far a graph is from a path

tw=1 pw=1

tw=3 pw=3

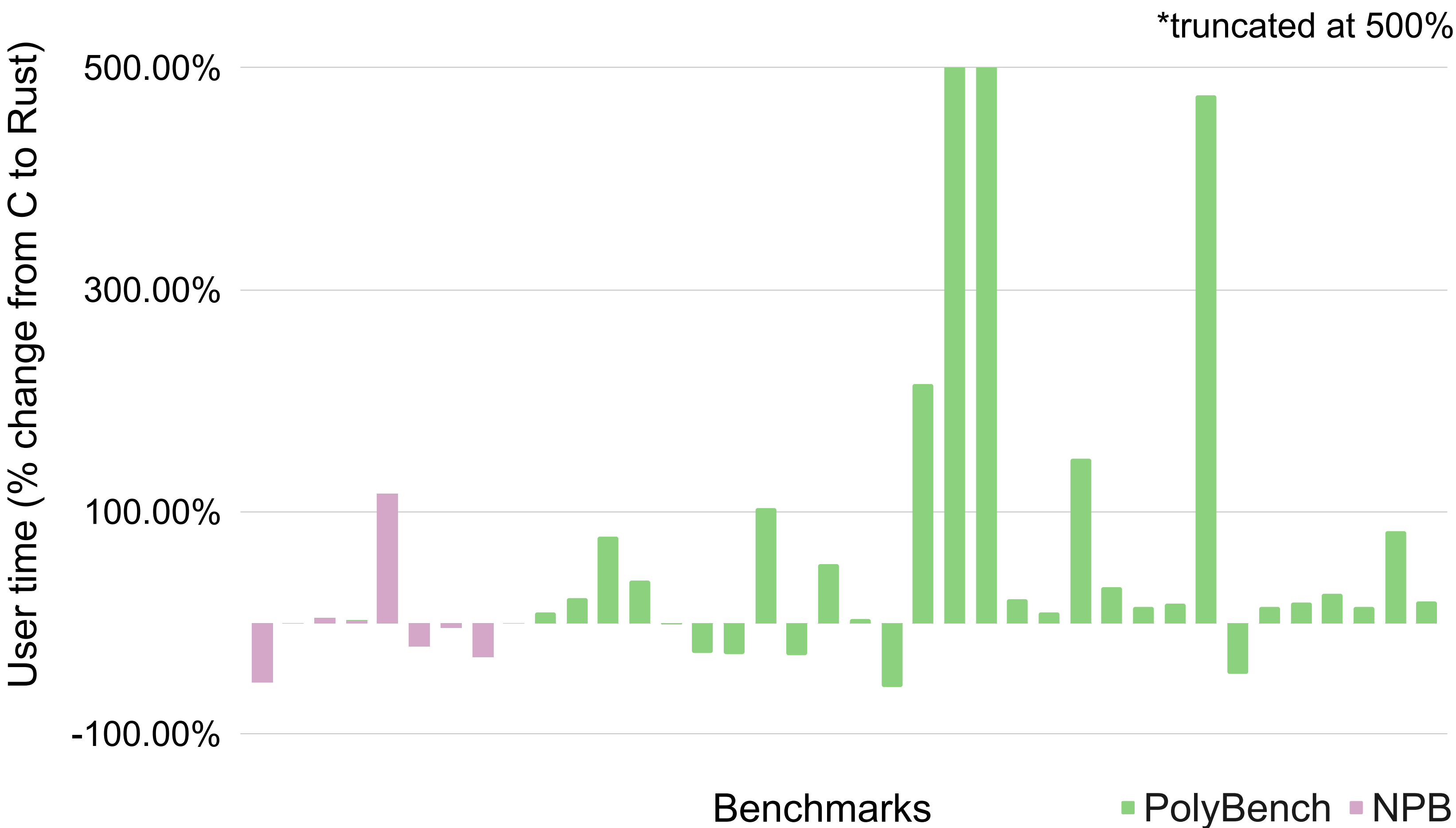


25x the vertices and 27x the edges!
Big graphs improve data flow analysis!

Rust def-use graphs also have
4x the treewidth and 47x the pathwidth!
That lets us parameterize algorithms!

IR differences should inform optimization!

Even with a “better” IR, there’s a performance gap



What other benchmarks or benchmark suites are worth exploring?

We can write property-aware algorithms

Static Single Information Form:
Since static information flows further, we can encode and use more of it to eliminate redundancies (e.g. bounds check deletion)

Improved Register Allocation:
High def-use graph pathwidth implies more simultaneously alive values, allowing greater allocation flexibility

Are there any other properties that could be interesting to track?